TRP23

TRAP MADE IT

MASTER VISION AND DEVELOPMENT PLAN

A team guide to the product vision, world design, technology, commerce, automation, roadmap and operating principles behind TRP23.

Internal Working Document

Version 1.0 | August 2026

THE LONG-TERM VISION MAY BE ENORMOUS. THE NEXT BUILD MUST STILL BE FOCUSED, PLAYABLE, MEASURABLE AND REAL.

1. Purpose of This Document

This document gives the entire TRP23 team one shared view of what we are building, why it matters, how the different parts connect, what should happen first, and what standards must guide every technical and creative decision.

It is deliberately ambitious. TRP23 is not being designed as a conventional fashion website, a small promotional game or a collection of disconnected features. The project is intended to grow into a living open-world platform in which entertainment, identity, progression, commerce, creativity, community and selected real-world actions can connect meaningfully.

At the same time, ambition without sequence will destroy the project. The team must protect the long-term vision while building focused, testable and genuinely enjoyable releases in the correct order.

> **Our first responsibility is not to build the entire metaverse. It is to create a vertical slice so strong that a player, collaborator, business or investor immediately understands why TRP23 could become enormous.**

How the team should use this plan

- Use it as the shared strategic reference for product, design, engineering, art, commerce and partnerships.
- Use it to challenge ideas that sound exciting but do not strengthen the next playable release.
- Use it to prevent conflicting assumptions between the existing web prototype, the Unity build, the backend and the wider commercial vision.
- Use it to create detailed audits, design documents, technical specifications, milestones and implementation tasks.
- Treat it as a living document. Changes should be recorded deliberately rather than quietly replacing the project's original purpose.

2. The Founding Vision

TRP23 / TRAP MADE IT / Trapology is intended to become far more than a normal game, clothing website or virtual shop.

> **A living digital world in which gameplay, identity, personal progress, commerce, community, creativity and selected real-world actions connect meaningfully.**

The initial commercial proposition is a clothing brand delivered through a game. The game should not merely advertise products. The world itself becomes the storefront, the story, the community and the experience.

What a player may eventually be able to do

- Explore a detailed recreation or interpretation of Lincoln, UK.
- Enter real or fictional shops and discover them naturally through the world.
- Inspect, try on and purchase physical clothing that can be delivered to their real address.
- Own, lease, operate or customise virtual commercial spaces.
- Sell approved real-world or digital products through those spaces.
- Build a legitimate virtual business and develop a reputation around it.
- Attend events, launches, collaborations and product drops.
- Take part in stories, missions and community challenges.
- Develop a persistent character, identity, history and reputation.
- Make choices that alter opportunities, relationships and parts of the environment.
- Connect selected achievements to useful real-world actions, training or opportunities.
- Move from a trap mindset towards ownership, accountability, discipline, creativity and opportunity.

Influences, not templates

The scale, immersion or systemic depth of games such as Grand Theft Auto, Red Dead Redemption, Cyberpunk 2077, The Elder Scrolls, Fallout, Watch Dogs, The Sims, Roblox, Fortnite, Minecraft, ARK, Rust, DayZ, No Man's Sky, Kingdom Come: Deliverance, Animal Crossing, Second Life and persistent roleplay servers can teach us important lessons.

We are not attempting to copy those games. We are studying why their strongest systems work, where they become repetitive or shallow, and how their underlying principles can be transformed into something distinctively TRP23.

3. The Non-Negotiable Purpose

The Trapology Bible and associated planning material define the deeper purpose of the project. That purpose must remain visible in the mechanics, consequences, progression and world design—not only in written dialogue or mission text.

The project should help players experience

- Recognising what traps a person.
- Accepting responsibility without humiliation.
- Separating circumstances from identity.
- Understanding choices and consequences.
- Replacing short-term survival thinking with long-term ownership.
- Creating legitimate opportunities.

- Developing standards, discipline and self-respect.
- Building routes towards creativity, employment, enterprise and community.
- Learning how to “untrap” themselves.

What TRP23 must not become

- A shallow crime fantasy.
- A police simulator framed around stereotypes.
- An endless glorification of dealing, violence or exploitation.
- A pay-to-win financial extraction system.
- A moral lecture disguised as a game.
- A patronising educational product.
- A generic corporate metaverse.
- An NFT or speculative-token scheme.
- A collection of disconnected mini-games.
- A shopping centre with walking controls.

The world may be raw, urban, funny, dramatic and culturally authentic. Players should be allowed to make poor decisions within appropriate fictional boundaries. The design should respond intelligently and truthfully, showing consequences, alternatives and routes forward rather than simply judging the player.

Eight questions for every major feature

1. Is it genuinely enjoyable?
2. Does it strengthen immersion or agency?
3. Does it fit the Trapology purpose?
4. Does it create meaningful choices?
5. Does it respect the player?
6. Can it be built and maintained?
7. Could it create legal, ethical, financial or safeguarding harm?
8. Does it move us towards the long-term vision without damaging the next playable release?

4. First Priority: Complete Repository Audit

Before broad implementation, the team needs one reliable, evidence-based understanding of what currently exists. The repository must be inspected recursively rather than understood through the README alone.

Areas that must be inspected

- Every Markdown document and the complete Trapology Bible.
- The original prototype and current Vite/Three.js frontend.
- Game, world, rendering, data and service modules.
- Admin, authentication, 2FA, commerce, wallet, bank, rewards, moderation and community systems.
- SQLite persistence and database assumptions.

- Map generation and Lincoln world tooling.
- Unity export and handoff scripts.
- Mobile packaging, deployment configuration and GitHub Actions.
- The complete Unity project: scenes, prefabs, scripts, ScriptableObjects, materials, shaders, packages and settings.
- Generated assets, abandoned experiments, duplicated systems and outdated documents.

Required audit outputs

Audit Area	Questions the team must answer
Executive assessment	What runs today? What is genuinely strong? What is prototype-only? What blocks the next demo and a secure release?
Architecture map	How do the browser client, Three.js runtime, Unity runtime, backend, database, admin, commerce, mobile and deployment layers connect?
Feature inventory	Which features are complete, working prototypes, partial, mocked, documented only, broken, duplicated, obsolete or unknown?
Technical debt and risks	Where are the security, architecture, data, build, testing, performance, licensing, privacy and maintenance risks?
Vision alignment	Where does the implementation express Trapology well, contradict it, glorify the trap or become too preachy?
Conflicting plans	Which documents and systems disagree, and what should become the source of truth?

Recommended source-of-truth categories

- Vision and doctrine
- Terminology
- World design
- Technical architecture
- Feature status
- Roadmap
- Content

- Commerce and currency
- Unity migration and ownership boundaries

Old documents should not be deleted automatically. Where necessary, they should be labelled as superseded so their history remains visible without confusing current development.

5. Define the Product Properly

TRP23 must be understandable at several levels: the first excellent playable experience, the persistent Lincoln world, connected physical commerce, the creator and business platform, and the eventual real-world impact platform.

The product thesis must answer

- What is TRP23 in one sentence?
- What is the player fantasy?
- What does a player do minute to minute?
- Why do they return tomorrow?
- What makes it different from GTA roleplay, Roblox, Second Life, Fortnite and a normal fashion store?
- Who is the first target player?
- What is the first market?
- What is the first commercially viable experience?
- What is the eventual platform vision?
- What is explicitly outside the first release?
- What must never be compromised?

The five product layers

Layer	Definition
1. First excellent playable experience	A focused, polished slice that a small team can genuinely finish.
2. Persistent Lincoln world	A growing urban open world with identity, businesses, missions and community.
3. Connected physical commerce	Real goods, delivery, inventory, returns, creators and approved merchants.
4. Creator and business platform	Player-operated spaces, tools, approved products, events and services.
5. Real-world impact platform	Opt-in connections between game activity, skills, opportunities, community actions and real

	outcomes.

Layer 5 must not be built before Layer 1 works. Each layer should create value in its own right and provide a stable foundation for the next.

6. Open-World Innovation Programme

The team should carry out a structured study of old and modern open-world games at the level of systems rather than surface features.

Systems to study

- World simulation and persistence
- NPC schedules and memory
- Emergent encounters
- Consequences, factions and reputation
- Economy and ownership
- Housing and businesses
- Survival and crafting
- Vehicles and traversal
- Multiplayer and social spaces
- Player-generated content
- Live events and procedural content
- Narrative choice and companions
- Law and consequence
- Roleplay and modding
- Commerce
- Accessibility
- Retention without manipulation

How each reference should be analysed

- What the system does exceptionally well.
- Why players respond to it.
- What eventually becomes repetitive or shallow.
- What TRP23 can learn.
- What TRP23 must avoid.
- How the underlying principle could be reimagined for this world.

Innovation output

The study should produce at least 50 original, properly defined ideas. Each idea should explain the player experience, mechanic, Trapology fit, required technology, complexity, business value, risk, earliest viable form and long-term version.

Ideas should be scored for fun, originality, Trapology alignment, feasibility, revenue potential, social value, legal complexity and operational burden. The final result should identify the top transformational ideas, low-cost/high-impact ideas, demo “wow” features, long-term moonshots and ideas that should be rejected despite sounding impressive.

7. Core Gameplay Vision

The game must remain enjoyable even when no purchase is made. Commerce can deepen the world, but it cannot be the only reason the world exists.

> **The core loop must never collapse into: walk to shop → view product → buy product.**

Proposed deeper loop

9. Explore the city.
10. Discover people, places, problems and opportunities.
11. Make decisions.
12. Develop skills, relationships, knowledge and resources.
13. Complete meaningful work, missions or creative challenges.
14. Change reputation and access.
15. Build or improve something.
16. Express identity.
17. Unlock new routes through the city and the player’s own case file.
18. Return to a world that remembers what happened.

Timescales the design must support

- 30-second loop
- 5-minute loop
- 30-minute loop
- Full session loop
- Daily loop
- Chapter loop
- Seasonal loop
- Long-term player journey

Progression beyond money and conventional XP

- Discipline
- Trust
- Craft
- Knowledge
- Influence
- Consistency
- Wellbeing
- Enterprise
- Community
- Creativity
- Street awareness
- Reliability
- Self-knowledge

This must not become a simplistic moral score. Traits should create trade-offs. Something useful in one situation may become harmful in another, and progression should reflect behaviour, consistency and relationships rather than a single “goodness” meter.

8. The Living Lincoln System

Lincoln must be designed as a living system rather than a photorealistic map with nothing meaningful happening inside it.

World-design questions

- How much geography should be accurate and how much should be compressed or fictionalised?
- Which landmarks are essential to recognition?
- Which permissions, licences and privacy considerations apply?
- How should roads, paths, waterways, interiors, public transport, weather, seasons and day/night operate?
- How will districts stream and expand?
- How will crowds, traffic, local events, business activity and NPC routines make the city feel alive?
- How will accessibility and future expansion beyond Lincoln be supported?

Repeatable world-data pipeline

- Unity as the production runtime.
- Appropriately licensed OpenStreetMap or GIS data.
- Terrain and elevation data.
- Modular building kits and spline-based roads.
- Procedural placement and district dressing.
- Addressable assets and additive scenes.
- Level of detail, occlusion, impostors and GPU instancing.
- Baked and dynamic lighting where appropriate.
- Photogrammetry only where lawful, licensed and genuinely useful.

- Automated validation of generated content.

External datasets should never be downloaded or integrated until their licences, attribution requirements and usage limitations have been documented. The goal is to build repeatable generation tools rather than manually place thousands of objects.

9. Real-World Commerce Inside the Game

A major long-term differentiator is the ability for an in-world product to correspond safely to a real physical product.

Target purchase journey

19. The player enters an in-game store.
20. The player examines or virtually tries on an item.
21. Current size, colour, stock, price and delivery region are retrieved from a commerce service.
22. The player adds the item to a secure basket.
23. The game clearly distinguishes game currency from real-money payment.
24. Checkout is handled by an appropriate regulated payment provider.
25. The order is recorded server-side and inventory is reserved safely.
26. Fulfilment receives the order.
27. Shipping and tracking appear in the player's account.
28. Returns and refunds are handled properly.
29. A virtual item may be granted separately under defined rules.

Production commerce requirements

- PCI scope minimisation and no storage of raw card details.
- Strong customer authentication.
- VAT, invoices and regional delivery rules.
- Stock reservation, idempotency and reconciliation.
- Refunds, returns, chargebacks, lost parcels and partial fulfilment.
- Fraud and account-takeover controls.
- Merchant payouts where later required.
- Customer support and consumer rights.
- Privacy, accessibility, staging, audit trails and error recovery.

> **The current mock commerce routes are development scaffolding. They must never be treated as production payment infrastructure, and every real-money action must be authoritative on the backend.**

10. Player-Operated Stores and Local Businesses

Virtual commercial spaces could eventually become one of TRP23's defining systems, but marketplace risk must be introduced in controlled stages.

Possible space categories

- TRAP MADE IT-owned stores
- Approved real Lincoln businesses
- Creator pop-ups
- Player-designed virtual-only shops
- Approved physical-goods sellers
- Event spaces
- Studios
- Barbers
- Galleries
- Music venues
- Training spaces
- Community organisations

Possible ownership and tenancy models

- Cosmetic lease
- Earned stewardship
- Limited-time pop-up
- Approved merchant tenancy
- Creator partnership
- Sponsorship
- Franchise or licensed presence
- Community-grant space

Systems required before open merchant selling

- Merchant onboarding and identity/business verification.
- Product approval, prohibited items and intellectual-property checks.
- Content moderation and product safety.
- Clear fulfilment, returns and tax responsibility.
- Commission and payout rules.
- Disputes, ratings and fraud detection.
- Shop customisation and analytics.
- Tenancy expiry and abandoned-shop management.
- Market saturation controls, discovery and accessibility.

The first demonstrable version should prove the experience of operating or visiting a store without yet exposing the project to uncontrolled marketplace payouts or arbitrary physical-goods selling.

11. Trap Coins and the Economy

An example exchange such as “£10 = 100 Trap Coins” is a concept, not an approved economy. The team must define exactly what each balance represents before money enters the system.

Balances that must remain distinct

- Soft currency earned through play
- Premium currency purchased with real money
- Loyalty rewards
- Store credit
- Discounts and promotional vouchers
- Withdrawable value
- Merchant balances
- Creator revenue
- Real-money purchases
- Non-transferable progression points

Economy questions

- Can purchased currency buy physical goods?
- Can it be transferred or gifted?
- Can it be earned or cashed out?
- Does it expire?
- How do refunds and chargebacks behave?
- What protections apply to minors?
- What spending limits and regional-pricing rules apply?
- How do Apple and Google rules affect the design?
- Could the system create money-laundering, gambling, loot-box or speculation risks?
- How are inflation, currency sinks, fraud and duplicated transactions controlled?

Ledger standard

Valuable balances should use a double-entry or equivalently robust immutable ledger. The client must never be trusted to change a balance. Every transaction should have a unique ID, player, type, amount, currency, source, destination, timestamp, idempotency key, status, reason, related order or activity and audit metadata.

> **The safest initial direction is a conventional, closed-loop and fully auditable game balance—not cryptocurrency, blockchain assets, NFTs or speculative investments.**

12. Real-Life Impact: Think Big, Build Carefully

"Everything you do in the game affects real life" should be interpreted broadly and responsibly. Real-world impact should not be reduced to spending money.

Potential opt-in connections

- Discovering and supporting local independent businesses.
- Booking real events or workshops.
- Finding apprenticeships or training.
- Learning practical skills.
- Submitting creative portfolios.
- Volunteering or supporting community projects.
- Verified discounts linked to constructive activity.
- Real-world scavenger trails and city events mirrored in-game.
- Fitness or walking challenges.
- Entrepreneurship and mentoring programmes.
- Creator collaborations and music showcases.
- Digital-to-physical product design.
- Community voting on real projects.
- Educational, charity and employment partnerships.

Safeguards for every real-world connection

- Consent
- Verification
- Cheating prevention
- Privacy
- Location safety
- Minors
- Accessibility
- Exclusion risk
- Partner responsibility
- Safeguarding
- Liability
- Manipulation risk
- Financial incentives
- Opt-out
- Data retention

No player should be punished for declining to link real-world accounts, health data, location data or personal activity.

13. AI-Driven and Procedural Automation

Automation is central to making the vision achievable for a small team. It should accelerate repeatable work while preserving human control over creative, financial, safeguarding and irreversible decisions.

World-generation automation

- Map-data import and validation
- Road and pavement generation
- Modular buildings and façade variation
- Interior shells
- Prop placement
- Vegetation
- Signage placeholders
- District dressing
- LOD generation
- Collider generation
- Navmesh preparation
- Occlusion setup
- Asset validation

Art-production assistance

- Concept exploration
- Texture and material variations
- Decals and signage drafts
- Clothing visualisation
- Animation cleanup and rig assistance
- Asset naming and metadata
- Thumbnail generation

Unlicensed AI-generated assets or assets with uncertain training-data provenance should not enter production without review.

Game design and narrative assistance

- Mission templates
- Branching-dialogue drafts
- NPC biographies
- Ambient conversations
- Event variants
- Balancing simulations
- Content linting
- Continuity checking
- Trapology alignment checks
- Repetition detection

AI-generated dialogue should not be shipped without human review.

Engineering and operations automation

- Boilerplate and tests
- API client generation
- Schema validation and migrations
- Static analysis and documentation
- Build and performance reports
- Asset dependency reports
- Scene validation and missing-reference scans
- Moderation triage
- Fraud signals
- Support categorisation
- Analytics summaries
- Merchant-onboarding assistance

> **AI must not make unreviewed irreversible moderation, financial, safeguarding or account-ban decisions.**

Required automation specification

Every automation should define its inputs, outputs, human review point, confidence checks, failure mode, rollback, cost, provider dependence, privacy implications and local/offline alternative where possible.

14. Unity Technical Direction

The existing Unity project must be inspected before architecture is chosen or replaced. Decisions should follow evidence rather than fashion.

What must be assessed

- Unity version
- Render pipeline
- Package health
- Input system
- Character controller and camera
- World streaming
- Scene structure
- Asset organisation
- Scripts and assemblies
- Save system
- API integration

- Build targets
- Performance profile
- Current map and free-roam state
- What should be retained, replaced or kept in the web prototype

Preferred long-term qualities

- Modular assemblies
- Clear domain boundaries
- Data-driven content
- ScriptableObjects where appropriate
- Server-authoritative valuable state
- Testable services
- Addressable assets
- Additive scene loading
- Object pooling
- Async loading
- Event-driven communication
- Dependency inversion
- Versioned save data
- Automated scene checks
- Profiling
- Scalable content pipelines

Large Unity packages should not be installed simply because they are popular. Each package requires a documented reason, licence, version, platform support, maintenance status, alternatives, lock-in risk and performance impact.

15. Graphics and Visual Quality Strategy

The visual goal is not simply “photorealism.” TRP23 needs a distinctive, achievable identity that scales across team size and target hardware.

Key choices

- URP versus HDRP
- Realistic versus stylised realism
- Baked versus dynamic lighting
- Day/night and weather
- Materials and decals
- Environmental storytelling
- Interiors
- Character fidelity
- Clothing simulation
- Reflections and post-processing
- Animation and facial systems
- Crowds and vehicles

- PC, console and mobile scalability

Quality tiers

- Developer blockout
- Prototype
- Vertical slice
- Public demo
- Production
- Showcase or cinematic

Automated visual validation

- Material validation
- Texture-size checks
- Missing LODs
- Collider checks
- Lightmap checks
- Shader compatibility
- Draw-call reports
- Overdraw warnings
- Asset naming
- Unused assets
- Missing references
- Scene budgets

Performance budgets should be measurable and tied to the first target hardware. Stable playability matters more than screenshots. Unless another target is deliberately approved, the design should favour a stable 60 FPS experience.

16. NPCs, Society and Emergent Systems

NPCs should participate in the world rather than stand still waiting to dispense missions.

Systems to explore

- Schedules
- Homes and workplaces
- Needs
- Memory
- Relationships
- Trust
- Rumours
- Social groups
- Businesses
- Personal goals

- Local knowledge
- Reactions to player history
- Procedural conversations
- Conflict and reconciliation
- Community events
- Mentorship
- Collaboration
- Economic behaviour

Four-tier NPC model

Tier	Purpose
1. Ambient population	Efficient crowds with simple schedules and reactions.
2. Local recurring NPCs	Named characters with memory, routines and relationships.
3. Story characters	Authored characters with deeper narrative arcs.
4. Dynamic agents	Carefully bounded simulation or AI-assisted behaviour.

Unrestricted player text must not be sent directly into an unmoderated language model. Any advanced agent system must define cost, latency, moderation, fallback behaviour and offline operation.

17. Missions That Express Trapology Through Play

Trapology should be experienced through choices, systems and consequences rather than explained through lectures.

Example mission structures

- Choosing between immediate cash and a longer-term opportunity.
- Repairing trust after failing a commitment.
- Building a legitimate event under pressure.
- Negotiating and solving a business problem.
- Helping someone without becoming responsible for their whole life.
- Identifying manipulation and setting boundaries.
- Learning a trade or managing limited resources.
- Recognising an escalating situation.
- Recovering after a setback and replacing a harmful routine.
- Collaborating with someone the player initially dislikes.

- Transforming an unused space.
- Investigating the player's own case file.

The mission system should avoid obvious "good choice / bad choice" dialogue wheels. It should support reusable mission grammars, state machines, failure recovery, consequence persistence and authoring tools.

18. Multiplayer and Community

TRP23 should not automatically become a massively multiplayer world. The right multiplayer structure must follow the first meaningful use case.

Models to compare

- Single-player with online services
- Small co-op
- Session-based multiplayer
- Community hubs
- Instanced districts
- Private servers
- Roleplay servers
- Shared persistent worlds
- MMO architecture

Areas that must be solved

- Authoritative server
- Client prediction and synchronisation
- Cheating and item duplication
- Voice and text moderation
- Harassment, blocking and reporting
- Minors and private spaces
- User-generated content
- Community events
- Server costs
- Scale testing
- Disaster recovery

> **A networking framework should not be selected until the team has defined the game's first real multiplayer use case.**

19. Safety, Ethics and Responsible Monetisation

TRP23 may appeal to young adults and potentially minors. Responsible design is therefore a product requirement, not a later legal checklist.

Core standards

- Age-appropriate experiences and parental consent where required.
- Spending limits and clear purchase confirmations.
- Transparent refunds and odds.
- No manipulative loot boxes, dark patterns or fake scarcity.
- No disguising real-money spending.
- No predatory streak systems or gambling-like cash-out loops.
- No exploitation of vulnerable users or targeted financial pressure.
- Clear advertising and sponsorship.
- Privacy by design and location safety.
- Strong reporting, harassment and moderation systems.
- Clear standards for self-harm, violence, drugs, criminal instruction, extremism and sexual content.
- Merchant-fraud prevention.

The project can be raw and culturally honest without being reckless.

20. Backend and Platform Architecture

The current backend should be treated as a mock foundation. The path to production should strengthen domain boundaries without prematurely splitting the system into dozens of microservices.

Potential bounded contexts

- Identity
- Player profile
- World state
- Progression
- Content
- Missions
- Inventory
- Catalogue
- Orders
- Fulfilment
- Payments
- Wallet
- Rewards
- Merchants
- Community

- Moderation
- Analytics
- Live operations

Platform requirements

- API versioning
- Schema validation
- Database choice and migrations
- Transactions and idempotency
- Queues and caching
- Object storage and CDN
- Authentication and authorisation
- Session management
- Rate limiting
- Audit logs
- Secret management
- Backups and restore testing
- Observability
- Privacy, deletion and data export
- Staging and deployment
- Feature flags and rollback

State the client must never control

- Currency
- Inventory ownership
- Purchases
- Rewards
- Merchant balances
- Verified progression
- Real-world fulfilment state

21. Analytics and Live Operations

Analytics should answer useful product questions without spying on players.

Useful events and measures

- Onboarding completion
- Mission starts, completions and abandonment
- Crashes and performance
- Navigation confusion
- Shop visits and product interest
- Basket conversion
- Payment failures and refunds
- Retention

- Community participation
- Moderation volume
- Economy health
- Exploit indicators

Live-operations capabilities

- Consistent event naming and schemas
- Dashboards
- Experiments
- Feature flags
- Content scheduling
- Seasonal events
- Rollback
- Incident response

Only necessary data should be collected, and consent requirements must be built into the design.

22. Practical Release Strategy

The roadmap is organised into horizons. Each horizon must produce a clear player-visible result and create the foundation for the next.

Horizon 0 — Stabilise and understand

- Complete the repository audit
- Verify builds
- Resolve architectural ambiguity
- Define sources of truth
- Establish tests and performance baselines

Horizon 1 — Playable vertical slice

- One polished Lincoln district
- Polished movement
- One complete chapter
- A meaningful case-file loop
- Several memorable NPCs
- One functioning shop
- One physical-product integration in sandbox/test mode
- Persistent progress
- One visually impressive exterior
- One high-quality interior
- A clear beginning and end
- Analytics
- Accessibility basics
- Reliable build and deployment

Horizon 2 — Closed community demo

- Accounts and secure saves
- Expanded district
- More missions
- Limited community features
- Real product fulfilment pilot
- Admin operations
- Moderation and support tools

Horizon 3 — Public early access

- Scalable backend
- Commerce operations
- Fraud controls
- Live events
- Content pipeline
- Performance coverage
- Reliable updates
- Customer support

Horizon 4 — Creator and merchant expansion

- Approved merchants
- Virtual spaces
- Creator tools
- Controlled marketplace
- Partnerships

Horizon 5 — Connected open-world platform

- Expanded city
- Multiplayer where justified
- Advanced simulation
- Broader real-world connections
- Additional cities or regions

Roadmap formats required

- 30-day action plan
- 90-day plan
- 12-month roadmap
- Long-term vision map

Every phase should define its objective, player-visible result, deliverables, dependencies, risks, acceptance criteria, tests, likely roles required and explicit “not included” list. Where evidence is insufficient, use effort bands and dependencies rather than false timeline precision.

23. Prioritisation Method

All proposed work should be scored consistently rather than selected by excitement alone.

Scoring factors

- Player value
- Vision alignment
- Technical necessity
- Commercial value
- Demo value
- Risk reduction
- Effort
- Dependencies
- Operational cost
- Reversibility

Decision categories

- Do now
- Do next
- Research
- Later
- Reject

> **We must avoid building glamorous systems on unstable foundations.**

24. Team Execution Rules

Once the audit and planning documents are complete, implementation should move in small, reviewable batches.

30. Select the highest-priority foundations that are safe to implement now.
31. Explain the intended change before making it.
32. Preserve working behaviour unless a documented roadmap deliberately replaces it.
33. Add or update tests.
34. Run the available checks and record the results.
35. Never claim a test passed unless it was actually run.
36. Do not silently rewrite founder doctrine or terminology.
37. Do not delete major systems without recording the reason and migration path.
38. Do not expose secrets or introduce paid services without approval.

39. Do not make real purchases or connect live payment accounts during development.
40. Do not introduce real-money transfers or deploy to production automatically.
41. Do not add unlicensed assets.
42. Do not fabricate research, benchmarks or legal conclusions.

Execution log

Each work session should record the date, objective, files inspected, findings, decisions, files changed, commands run, test results, unresolved issues and recommended next action. This prevents knowledge being lost between tools, developers or AI-assisted sessions.

25. Initial Engineering Work to Investigate

The following items are strong candidates for early work, but each must first be confirmed by the audit.

- Define a clear boundary between the Three.js prototype and the Unity production path.
- Create a shared, versioned content schema.
- Generate Unity ScriptableObjects or JSON assets from a canonical content source.
- Replace mock-value assumptions with explicit environment modes.
- Harden wallet-ledger rules.
- Add proper database migrations.
- Add commerce and wallet contract tests.
- Add automated Unity scene and prefab validation.
- Add missing-reference and collision checks.
- Improve map-generation reproducibility.
- Create the first small vertical-slice scene.
- Create a Unity API-client abstraction.
- Implement secure development authentication.
- Make catalogue data server-driven.
- Create clear sandbox-checkout boundaries.
- Introduce addressables and additive world loading where justified.
- Document asset and performance budgets.
- Generate architecture diagrams where practical.
- Create issue-ready tasks from the roadmap.

26. Required Team Outputs

At the end of the first full analysis and planning cycle, the team should be able to read one founder-facing summary that answers the following clearly.

- What TRP23 is.
- What currently exists, separated into genuine implementation, mock systems and plans.
- The five strongest existing foundations.

- The ten most important problems, ranked.
- The single best vertical slice, described from launch to completion.
- The ten strongest future innovations.
- What should be automated and what must remain under human control.
- What should be built next, in order.
- What must wait.
- Which founder decisions genuinely cannot be resolved from evidence or technical judgement.
- The first implementation batch, including files affected, risks and verification commands.

27. Quality Standard

Every recommendation, design and implementation decision must be specific to TRP23, evidence-based, commercially aware, technically realistic, creatively ambitious and honest about uncertainty.

The work must be

- Grounded in the Trapology Bible.
- Designed for a small team that may grow.
- Protective against premature complexity.
- Structured so another developer can continue it.
- Capable of producing visible progress rather than endless documentation.

Empty language is not enough

Phrases such as “use AI to improve NPCs,” “add blockchain,” “make it like GTA,” “create an immersive metaverse,” “use procedural generation,” “add realistic graphics” or “build a scalable backend” are not plans.

Every recommendation must explain what it is, why it matters, how it works, when it belongs in the roadmap, what it depends on, what could go wrong, what the earliest useful version looks like and how the team will prove that it works.

> **Think as ambitiously as the strongest studios and creator platforms. Execute with the discipline of a small team that cannot afford wasted months.**

28. Immediate Shared Direction

The first collective task is to inspect the entire repository and create the master repository audit. Broad implementation should not begin until the team can

distinguish the existing working product from mocks, experiments, duplicated systems and future concepts.

The first team sequence

43. Complete the repository audit.
44. Agree the product thesis and source-of-truth documents.
45. Define the first vertical slice.
46. Approve the technical boundary between the web prototype, Unity and backend.
47. Turn the roadmap into issue-ready work packages.
48. Begin the first small implementation batch.
49. Test, record and review every batch.
50. Keep the vision large and the next build focused.

TRP23 is not simply a game with a shop inside it. It is a world in which entertainment, identity, enterprise, community and real opportunity can eventually meet. Our job now is to prove that future through one exceptional, buildable step at a time.